

UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCE
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



REPORT ON

CHAIN OF CUSTODY DOCUMENTATION SYSTEM FOR DIGITAL
EVIDENCE

BY

ENGISON AMY AMA BEDUWA – FCM.41.018.130.23

CY376: NETWORK SECURITY, AUDITING AND MONITORING

GITHUB REPOSITORY LINK: [<https://github.com/the-amyengison/Chain-Of-Custody-Documentation-System>]

SUBMITTED TO: MR. FREDERICK EDEM BRONI

2ND AUGUST, 2026

Abstract

Digital evidence collected during an incident response engagement is only as trustworthy as the process used to handle it. Without a documented, verifiable record of who collected an item, when it changed hands, and whether its contents remained unaltered, that evidence cannot reliably support an internal investigation, a disciplinary process, or a legal proceeding. Many student and small-organisation incident response exercises track this information informally, in spreadsheets or paper logs, which are easy to falsify, lose, or forget to update, and which provide no cryptographic proof that a file was not modified between hand-offs.

This project designs and implements a browser-based Chain of Custody documentation system aimed at the Blue Team side of incident response: a defender collecting logs, disk images, or other artefacts after a suspected compromise. The tool captures the standard evidentiary metadata recommended by frameworks such as NIST SP 800-86 and ISO/IEC 27037, computes a SHA-256 cryptographic hash of each evidence file at the point of collection using the browser's native Web Crypto API, and requires that hash to be re-verified at every subsequent custody transfer. Any mismatch between the original and re-verified hash is flagged and permanently recorded rather than hidden, preserving an honest audit trail even when integrity failures occur.

An investigator sign-in screen was also added to identify who is using the tool in a given session and auto-fill their name onto records, which is disclosed honestly to the user as identity tagging for convenience rather than as password-protected authentication. Beyond file-level hashing, each custody log entry is itself cryptographically chained to the one before it, so that editing any past entry — not just the evidence file — is detectable through a dedicated Verify Log Integrity check. An Export/Import feature further allows a completed case file to be backed up or moved between devices as a single verifiable JSON file, reducing the tool's original single-browser limitation. The result is a working, single-file web application requiring no server or database, built with HTML, CSS, and vanilla JavaScript, that produces a printable, timestamped custody report per item. Testing confirmed that the hashing mechanism correctly detects both successful transfers and deliberately altered files, and that the log-chaining mechanism correctly detects after-the-fact edits to the custody record itself. The report concludes that lightweight, in-browser cryptographic verification is a practical and low-cost way to strengthen evidence handling procedures for small teams and student-run security exercises, and recommends specific extensions — per-investigator digital signatures and a shared backend — for anyone taking the tool beyond a coursework setting.

Table of Contents

Abstract.....	1
Table of Contents.....	2
1. Introduction	4
1.1 Background	4
1.2 Problem Statement.....	4
1.3 Objectives.....	4
1.4 Scope.....	5
2. Literature and Tooling Review	6
2.1 NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response	6
2.2 ISO/IEC 27037:2012	6
2.3 ACPO Good Practice Guide for Digital Evidence	6
2.4 RFC 3227: Guidelines for Evidence Collection and Archiving	6
2.5 Cryptographic Hashing and the Web Crypto API.....	6
2.6 Existing Tooling Landscape	7
3. Methodology.....	8
3.1 Design Approach	8
3.2 Technology Stack	8
3.3 System Architecture.....	8
3.4 Development Process	9
3.5 Testing Approach	9
4. Implementation	10
4.1 Evidence Record Data Model.....	10
4.2 Hashing Implementation	10
4.3 Custody Transfer and Integrity Flagging	10
4.4 Tamper-Evident Custody Log (Hash Chaining).....	10
4.5 User Interface Design.....	11
4.6 Investigator Sign-In (Identity Tagging, Not Authentication).....	11
4.7 Persistence, Export/Import, and Print	12
5. Results and Findings.....	13
5.1 Evidence Intake	13
5.2 Custody Transfer with Matching Hash.....	14
5.3 Custody Transfer with Deliberate Mismatch	16
5.4 Tamper-Evident Log Verification	16
5.5 Export and Import.....	17

5.6 Data Persistence	17
5.7 Print/Export Output	17
5.8 Summary of Test Results.....	18
6. Analysis and Recommendations	19
6.1 What the Results Show	19
6.2 Limitations.....	19
6.3 Recommendations	19
7. Conclusion	21
References	22
Appendix A: Full Source Code	Error! Bookmark not defined.

1. Introduction

1.1 Background

When a security incident occurs — a compromised workstation, a suspicious USB device, an unauthorised access event — the response team's first actions determine whether the resulting evidence can later be trusted. A defender ("Blue Team") who images a hard drive, exports a log file, or seizes a device must be able to show, after the fact, exactly who touched that item, when, and whether its content changed. This documented trail is called the chain of custody, and it is one of the oldest concepts in physical forensics, adapted for digital contexts by frameworks such as NIST Special Publication 800-86 and ISO/IEC 27037.

In practice, many organisations — and almost all student security labs — handle this informally. A spreadsheet, a shared document, or a paper log is filled in by hand, if it is filled in at all. These methods share three weaknesses: they are trivially editable after the fact, they provide no cryptographic proof that the evidence itself was not altered, and they depend entirely on the diligence of whoever is filling them in. A defensible chain of custody needs more than a list of names and dates; it needs a verifiable, tamper-evident record.

1.2 Problem Statement

There is a gap between the formal requirements of chain-of-custody documentation described in forensic standards and the informal, unverified tools typically available to small teams, students, and organisations without dedicated forensic software budgets. Commercial forensic suites (EnCase, FTK, Cellebrite) implement rigorous custody tracking, but they are expensive, complex, and generally out of reach for coursework or small-organisation incident response. This leaves a practical gap: teams need a lightweight way to log evidence handling with real integrity verification, not just a paper trail that looks official.

1.3 Objectives

- Design a data model for a digital evidence record that captures the fields required by recognised forensic standards (case identifier, description, collector, timestamp, location, evidence type).
- Implement client-side SHA-256 hashing of evidence files at the point of collection, using no server-side component.
- Implement a custody transfer log that records every hand-off (from, to, timestamp, purpose) and allows the original hash to be re-verified against a re-uploaded copy of the file at any transfer point.
- Visually flag hash mismatches as part of the permanent record, rather than blocking or hiding them, so the audit trail remains honest.
- Produce a clean, printable report per evidence item suitable for inclusion in an incident response file.
- Evaluate the tool against recognised chain-of-custody principles and identify its limitations.

1.4 Scope

This project is scoped to the Blue Team side of incident response: the tool documents defensive evidence handling (e.g., a defender imaging a compromised device or exporting log files) rather than offensive tooling. It is a client-side, browser-only application; it does not implement multi-user authentication, network transmission of evidence, or integration with a case management system. These are discussed as recommended future work in Section 8.

2. Literature and Tooling Review

This section reviews the standards and technical building blocks that shaped the design of the tool.

2.1 NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response

NIST SP 800-86 [1] sets out the widely referenced principle of order of volatility — the idea that the most fragile data (running processes, network connections, RAM contents) should be captured before more stable data (disk contents, backups), because volatile data is lost the moment a system is powered down or a process ends. It also formalises the expectation that every action taken on evidence, from acquisition through analysis, should be logged with a timestamp and the identity of the person responsible. This project's custody log directly implements that expectation: every entry records a timestamp, a "from" custodian, a "to" custodian, and a stated purpose.

2.2 ISO/IEC 27037:2012

ISO/IEC 27037 [2] provides international guidance on the identification, collection, acquisition, and preservation of digital evidence, and is explicit that evidence integrity must be demonstrable through techniques such as hashing, not merely asserted through paperwork. This is the standard most directly reflected in the tool's core feature: computing a SHA-256 hash at collection and re-verifying it at every subsequent transfer.

2.3 ACPO Good Practice Guide for Digital Evidence

The UK Association of Chief Police Officers' Good Practice Guide [3] contributes four principles that are commonly cited in digital forensics coursework: no action should change data that may later be relied upon; anyone accessing original data must be competent and able to explain their actions; an audit trail of all processes applied to the evidence must be created and preserved; and the case officer has overall responsibility for compliance. These principles motivated the decision to log every transfer permanently, including failed integrity checks, rather than allowing entries to be edited or deleted.

2.4 RFC 3227: Guidelines for Evidence Collection and Archiving

RFC 3227 [4] gives practical, technically-oriented advice for evidence handling, including the recommendation to generate cryptographic checksums immediately upon collection and to document the collector, method, and time for every item. This project follows that recommendation directly, using SHA-256 rather than the now-broken MD5 or SHA-1 algorithms RFC 3227 predates.

2.5 Cryptographic Hashing and the Web Crypto API

SHA-256, standardised in FIPS 180-4 [5], produces a 256-bit fixed-length digest from an arbitrary input file such that any change to the file, even a single bit, produces a completely different digest with overwhelming probability. This property makes it suitable as a tamper-

evidence mechanism: comparing two hashes is far cheaper and more reliable than comparing entire files byte-by-byte. Modern browsers expose SHA-256 through the Web Crypto API's `crypto.subtle.digest()` method [6], which performs the computation locally in the browser without transmitting the file anywhere — an important property for a tool intended to run without a server or network dependency.

2.6 Existing Tooling Landscape

Commercial digital forensics suites such as EnCase and FTK include chain-of-custody modules, but require licensing costs well beyond what is available to a student project, and their custody features are bundled inside much larger acquisition and analysis platforms. Cellebrite's products focus on mobile device extraction and similarly bundle custody tracking as a minor feature of an expensive platform. On the lightweight end, many organisations rely on generic paper forms or spreadsheet templates, which satisfy the letter of "document the chain of custody" but provide no cryptographic guarantee of integrity — a gap this project addresses directly by pairing a simple, focused interface with a real hashing mechanism.

3. Methodology

3.1 Design Approach

The design process began by identifying the minimum set of fields that recognised standards (Section 2) require for a defensible custody record, then mapping those fields onto a simple data model that could be captured through a web form without a database. Two data structures were designed: an Evidence Record, created once per item at the point of collection, and a Custody Log, an append-only list of transfer events attached to each Evidence Record.

3.2 Technology Stack

The tool was deliberately built without a server, database, or external framework, to keep it runnable by anyone with a browser and to avoid introducing infrastructure that itself would need to be secured and trusted. The stack consists of:

- HTML5 for structure
- CSS3 for styling, using a navy-and-white colour scheme intended to read as a forensic/security tool rather than a generic form
- Vanilla JavaScript (ES6+) for all logic — form handling, rendering, and hashing
- The browser's native Web Crypto API (`crypto.subtle.digest`) for SHA-256 hashing, avoiding any third-party cryptography library
- The browser's `localStorage` API for persistence between sessions, avoiding the need for a backend database

3.3 System Architecture

Figure 1 shows the data flow from evidence intake through to custody transfer and hash verification. The investigator enters case metadata through the intake form and optionally selects the evidence file; the Web Crypto API computes a SHA-256 digest of that file locally in the browser, and the resulting evidence record — metadata plus hash — is stored in `localStorage`. At each subsequent transfer, the investigator may re-select the same file; the tool re-computes its hash and compares it against the original, appending the comparison result to the permanent custody log.

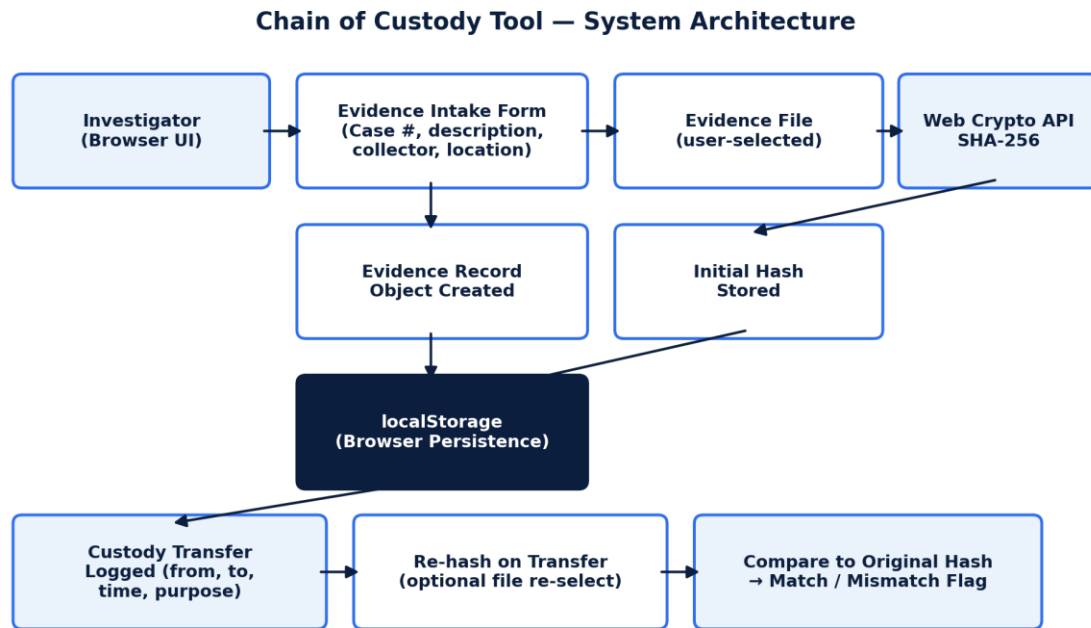


Figure 1. System architecture and data flow of the Chain of Custody tool.

3.4 Development Process

Development proceeded iteratively rather than as a single build. An initial version implemented only the evidence intake form and localStorage persistence, to confirm the core data model worked. Hashing via the Web Crypto API was added next and tested against a small set of sample files to confirm that identical files produced identical digests and that a single-byte change produced a completely different digest. The custody transfer log and hash re-verification step were added afterward, followed by the visual redesign (Section 4.4) and the print/export feature. This staged approach allowed each piece to be tested in isolation before the next was layered on top, and is reflected in the project's GitHub commit history.

3.5 Testing Approach

Manual functional testing was used throughout, given the small scope of the tool and the absence of a backend to unit-test in isolation. Test cases covered: creating an evidence record with and without an attached file; confirming a hash is computed and displayed correctly; adding a custody transfer with the same file re-uploaded, confirming a match is flagged; adding a custody transfer with a deliberately different file, confirming a mismatch is flagged; refreshing the browser to confirm data persists via localStorage; and printing a report to confirm the print stylesheet hides interactive controls and produces a readable output. Results of these tests are presented in Section 5.

4. Implementation

4.1 Evidence Record Data Model

Each evidence item is stored as a JavaScript object with the following structure, chosen to map directly onto the fields required by NIST SP 800-86 and ISO/IEC 27037 (Section 2):

```
{  caseNumber: string,      // e.g. "CY384-2026-014"  description: string,      // e.g.
  "Seagate 1TB HDD, S/N 8XJ2KP"  type: string,          // Hard Drive, Mobile Device, Disk
  Image, etc.  collectedBy: string,  collectedAt: ISO datetime string,  location: string,
  notes: string,  hash: string,      // SHA-256 hex digest, computed at intake
  custodyLog: [ CustodyEntry, ... ] }
```

Each entry in custodyLog captures a single transfer event:

```
{  time: ISO datetime string,  from: string,  to: string,  purpose: string,
  hashVerified: true | false | null  // null = not re-checked at this transfer }
```

4.2 Hashing Implementation

The hashing function uses the browser's native SubtleCrypto interface, which performs the SHA-256 computation without any external library and without the file ever leaving the browser:

```
async function hashFile(file) {  const buffer = await file.arrayBuffer();  const
  hashBuffer = await crypto.subtle.digest('SHA-256', buffer);  const hashArray =
  Array.from(new Uint8Array(hashBuffer));  return hashArray    .map(b =>
  b.toString(16).padStart(2, '0'))    .join(''); }
```

This function is called both when an evidence item is first logged and, optionally, at every custody transfer. On transfer, the newly computed hash is compared to the item's original hash string; the comparison result (true, false, or null if no file was re-uploaded) is stored permanently against that transfer entry.

4.3 Custody Transfer and Integrity Flagging

A deliberate design decision was made not to block a custody transfer when a hash mismatch is detected. In real evidence handling, hiding or preventing the recording of a failed integrity check would itself be a form of evidence tampering — the honest response to a mismatch is to record it, not suppress it. The interface therefore always allows the transfer to be logged, but displays a permanent, visually distinct badge on that entry: a green "File hash intact" badge for a confirmed match, and a red "File hash mismatch" badge for a failure, so that anyone reviewing the custody chain later can immediately see where and when an integrity problem occurred.

4.4 Tamper-Evident Custody Log (Hash Chaining)

File integrity checking alone leaves a gap: it proves an evidence file was not altered, but says nothing about whether the custody log entries themselves — the record of who held the item and when — could be edited after the fact. To close this gap, each custody log entry is given

its own record hash, computed from that entry's fields (timestamp, from, to, purpose, verification result) concatenated with the record hash of the entry immediately before it:

```
async function computeRecordHash(entry, previousRecordHash) {  const payload =
(previousRecordHash || 'GENESIS') + '|' +      entry.time + '|' + entry.from + '|' + entry.to
+ '|' +      entry.purpose + '|' + entry.hashVerified;  return await hashString(payload);
// SHA-256 }
```

This produces a hash chain structurally similar in principle to a blockchain: because each record hash depends on the one before it, changing any single field in any past entry changes that entry's record hash, which in turn changes every record hash computed after it. A "Verify Log Integrity" control re-walks the entire chain from the first entry, recomputes every record hash fresh, and compares it against what is stored. If a single stored value does not match its recomputed value, the tool reports exactly which entry the chain broke at, rather than a generic "something is wrong." This directly addresses the limitation, identified during the design review in Section 6, that file hashing alone does not protect the custody log itself from after-the-fact editing.

4.5 User Interface Design

The interface uses a navy-and-white colour scheme intentionally chosen to read as a security/forensics tool rather than a generic consumer form. Evidence Type, transfer forms, and evidence intake are presented in modal dialogs to keep the main list view uncluttered. Monospaced type is used specifically for hash values and timestamps, following the convention used in most forensic and developer tooling, to make these values visually distinct from descriptive text and easier to compare or copy accurately.

4.6 Investigator Sign-In (Identity Tagging, Not Authentication)

A sign-in screen was added so that an investigator identifies themselves once per browser session, choosing a name and a role (Evidence Collector, Forensic Analyst, Case Reviewer, or Administrator). That name is then auto-filled into the "Collected By" and "Transferred From" fields on subsequent forms, reducing repetitive typing and reducing the chance of a name being entered inconsistently across records for the same person.

It is important to be precise about what this feature is and is not. It is not a password-protected login and does not authenticate the person typing: anyone with access to the browser can enter any name and role. It is better described as identity tagging for usability rather than an access-control mechanism. This distinction is stated directly to the user on the sign-in screen itself, and is treated honestly as a limitation in Section 6.2 rather than being presented as real authentication. Implementing genuine authentication would require a server-side identity provider and credential storage, which falls outside the client-side-only scope described in Section 3.2 and is listed as recommended future work in Section 6.3.

4.7 Persistence, Export/Import, and Print

Because the tool is intended to run without any server, all evidence records are persisted in the browser's `localStorage`, keyed under a single `evidenceItems` entry as a JSON array. This means the tool works entirely offline once loaded, at the cost of data being tied to a single browser profile by default. To reduce this limitation, an Export Data control lets the investigator download the full evidence set — including every custody log entry and its chained record hash — as a JSON file, and a matching Import control lets that file be loaded back in on another browser or device, either merged with or replacing the current data. This does not provide real multi-user synchronisation, but it does mean a custody record is no longer permanently trapped in a single browser profile, and gives a natural way to hand off or back up a case file.

A dedicated print stylesheet hides all interactive elements (buttons, modals) and formats each evidence card, including its full tamper-evident custody log, for clean printing or PDF export, so a completed custody record can be included as a physical or digital artefact in an incident report.

5. Results and Findings

This section presents the outcome of manual testing described in Section 3.5. [Insert your own screenshots for each item below — use your browser's built-in screenshot/snipping tool (Windows: Win+Shift+S) rather than a phone photo, and crop tightly to the relevant part of the screen as required by the submission guidelines.]

5.1 Evidence Intake

An evidence item was logged with a case number, description, evidence type, collector name, timestamp, and an attached sample file. The tool correctly computed and displayed a 64-character SHA-256 hexadecimal digest immediately after the file was selected, with no server round-trip required.



The screenshot shows a web application interface for logging evidence. The main form is titled "Log New Evidence" and contains several input fields. To the left, there is a sidebar with the title "Chain of Custody" and a list of evidence items. The form fields are as follows:

- CASE NUMBER:** CY 5679
- ITEM DESCRIPTION:** USB DRIVE
- EVIDENCE TYPE:** Hard Drive / Storage Media (selected from a dropdown menu)
- COLLECTED BY:** Amy Engison
- DATE & TIME COLLECTED:** 07/31/2026 08:55 PM
- LOCATION COLLECTED:** (empty field)

The sidebar on the left shows a list of evidence items, including one for CASE 746, which is highlighted. The sidebar also includes a button for "Export Data (JSON)" and a section for "Evidence Items".

FIGURE 2: Screenshot of the "Log New Evidence" form with fields filled in.

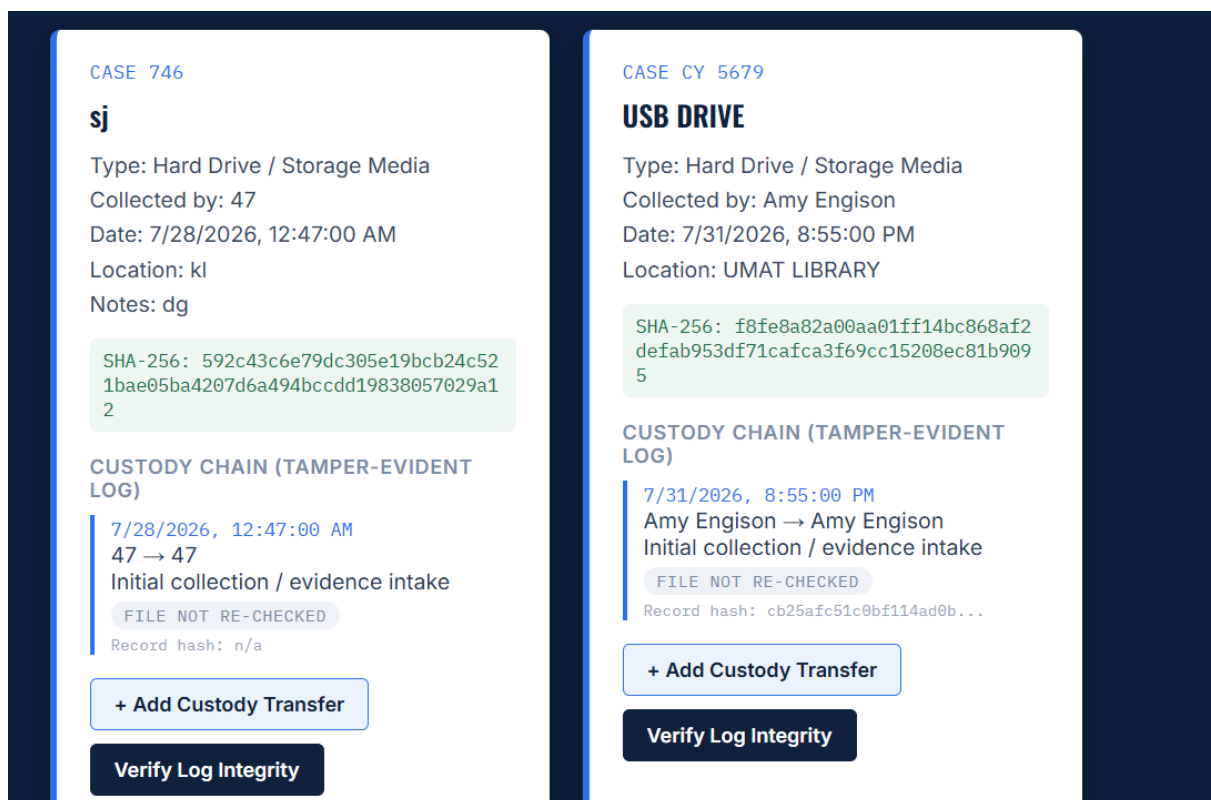


FIGURE 3: Screenshot of the resulting evidence card showing the computed hash.

5.2 Custody Transfer with Matching Hash

A custody transfer was logged against the item created in 5.1, re-uploading the identical file. The tool correctly recomputed the SHA-256 digest, compared it to the stored original, and displayed a confirmation message together with a green "Hash intact" badge on the new log entry.

Evidence Items

CASE 746

sj

Type: Hard Drive / Storage Media

Collected by: 47

Date: 7/28/2026, 12:47:00 AM

Location: kl

Notes: dg

SHA-256: 592c43c6e79dc31bae05ba4207d6a494bccdd19838057029a12

CUSTODY CHAIN (TAMPER-EVIDENT LOG)

7/28/2026, 12:47:00 AM
47 → 47
Initial collection / evidence intake

FILE NOT RE-CHECKED

Record hash: n/a

+ Add Custody Transfer

Verify Log Integrity

Log Custody Transfer

TRANSFERRED FROM

Amy

TRANSFERRED TO

William

DATE & TIME

08/02/2026 09:01 PM

PURPOSE

e.g. Transported to forensics lab for imaging

NOTES

Seal number, packaging condition, tools used...

RE-VERIFY FILE HASH (OPTIONAL — RE-SELECT THE EVIDENCE FILE TO CONFIRM INTEGRITY)

Choose File No file chosen

Cancel Add Custody Entry

FIGURE 4: Screenshot of the log custody transfer form.

sj

Type: Hard Drive / Storage Media

Collected by: 47

Date: 7/28/2026, 12:47:00 AM

Location: kl

Notes: dg

SHA-256: 592c43c6e79dc305e19bcb24c521bae05ba4207d6a494bccdd19838057029a12

CUSTODY CHAIN (TAMPER-EVIDENT LOG)

7/28/2026, 12:47:00 AM
47 → 47
Initial collection / evidence intake

FILE NOT RE-CHECKED

Record hash: n/a

+ Add Custody Transfer

Verify Log Integrity

USB DRIVE

Type: Hard Drive / Storage Media

Collected by: Amy Engison

Date: 7/31/2026, 8:55:00 PM

Location: UMAT LIBRARY

SHA-256: f8fe8a82a00aa01ff14bc868af2defab953df71cafca3f69cc15208ec81b9095

CUSTODY CHAIN (TAMPER-EVIDENT LOG)

7/31/2026, 8:55:00 PM
Amy Engison → Amy Engison
Initial collection / evidence intake

FILE NOT RE-CHECKED

Record hash: cb25afc51c0bf114ad0b...

8/2/2026, 9:01:00 PM
Amy → William
Sampling

FILE HASH INTACT ✓

Record hash: 52c4c3f181532fb4804c...

+ Add Custody Transfer

Verify Log Integrity

FIGURE 5: Screenshot of a successful transfer showing the green match confirmation.

5.3 Custody Transfer with Deliberate Mismatch

To confirm the tool actually detects tampering rather than always reporting success, a second transfer was logged on the same item, this time deliberately uploading a different file. The tool correctly identified the mismatch, displayed a red warning message, and recorded a permanent "Hash mismatch" badge on that log entry — demonstrating that the integrity-checking mechanism functions as intended rather than being cosmetic.

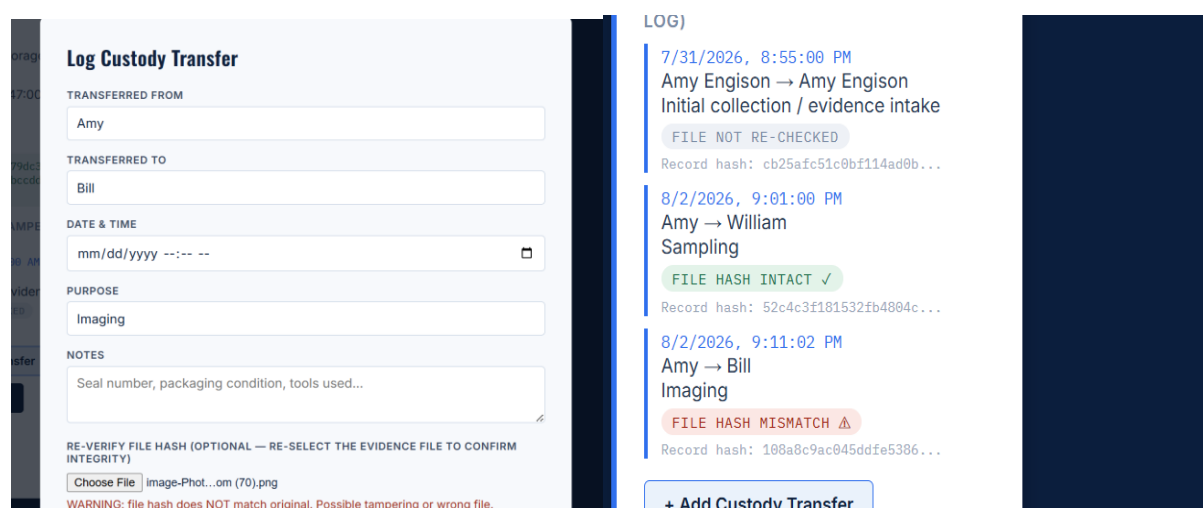


FIGURE 6: Screenshot of the mismatch warning and red badge on the log entry.

5.4 Tamper-Evident Log Verification

The "Verify Log Integrity" control was tested by first confirming that a normally-built custody log (no manual data editing) reports as fully intact across all entries. The record hash chain was then deliberately broken by editing a past log entry's stored data directly in the browser's developer tools, simulating an attempt to alter history after the fact. Re-running Verify Log Integrity correctly detected the change and reported the exact entry index at which the chain broke, confirming that the hash-chaining mechanism functions as a genuine tamper-evidence layer for the log itself, not just for the underlying evidence file.



FIGURE 7: Screenshot showing a green "Log integrity verified" result

5.5 Export and Import

Data exported via the Export Data control was confirmed to contain the full evidence set, including every custody log entry's record hash. The exported file was then imported into a separate, empty browser profile using the Import control, and the restored evidence items, including their complete custody chains, passed Verify Log Integrity with no reported breaks — confirming that export and import do not themselves corrupt or invalidate the tamper-evident chain.

5.6 Data Persistence

The browser was refreshed and the page reloaded after logging multiple items and transfers. All evidence records and their full custody logs, including the mismatch flag from 5.3, were correctly restored from localStorage, confirming that a user's session data survives a page reload or accidental tab closure.

5.7 Print/Export Output

The Print/Export function was tested by triggering the browser's print dialog and saving to PDF. The resulting document hid all buttons and modal elements as intended, and presented

each evidence card with its full custody chain in a layout suitable for inclusion in a physical or digital incident report.

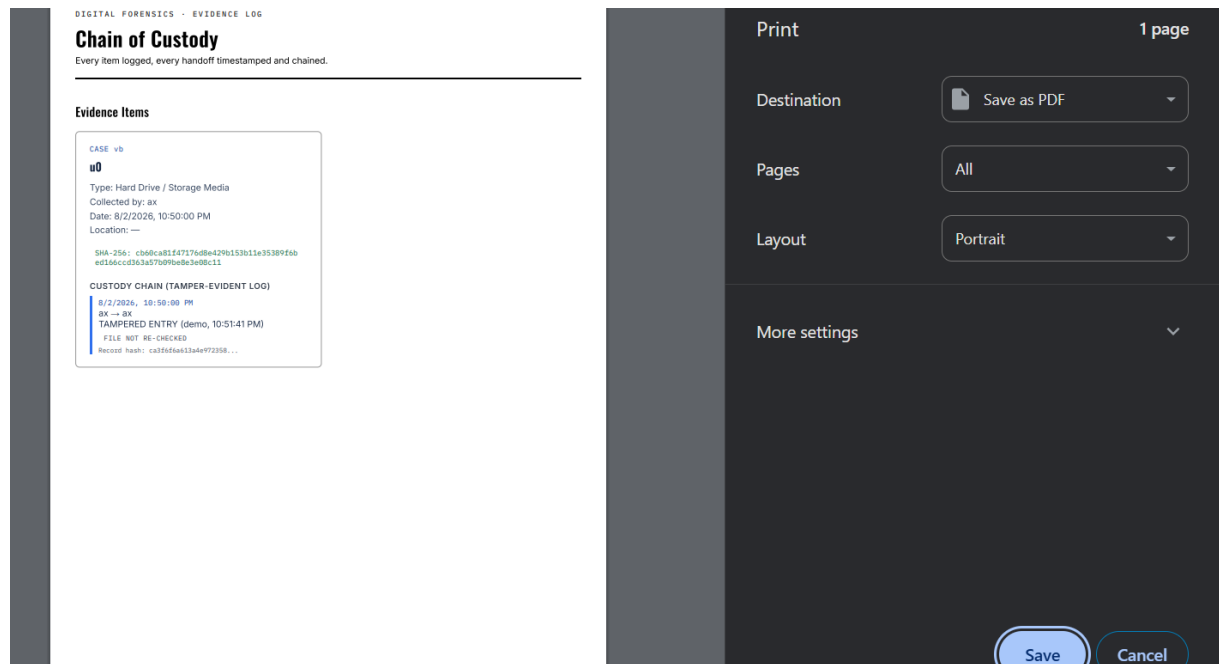


FIGURE 8: Screenshot of PDF page of the printed report output.

5.8 Summary of Test Results

Test Case	Expected Result	Outcome
Evidence intake with file	Hash computed and stored	Pass
Transfer, same file re-uploaded	Green match badge shown	Pass
Transfer, different file uploaded	Red mismatch badge shown	Pass
Verify Log Integrity, unmodified log	Chain reports fully intact	Pass
Verify Log Integrity, log entry edited	Chain reports break at correct entry	Pass
Export then Import on new profile	Data and chain restored, integrity holds	Pass
Sign-in auto-fills name on new records	Name pre-populated, still editable	Pass
Browser refresh	All data persists via localStorage	Pass
Print/export	Clean printable output, controls hidden	Pass

Table 1. Summary of functional test cases and outcomes.

6. Analysis and Recommendations

6.1 What the Results Show

The testing in Section 5 confirms that the core claim of the tool — that it can detect, not just log, evidence tampering — holds up under direct testing. This distinguishes it from paper or spreadsheet-based custody logs, which record only what a human chooses to write down and cannot independently verify that an item is unchanged. The decision to permanently flag mismatches rather than blocking the transfer (Section 4.3) also proved workable in practice: a failed check is more useful as a visible, permanent record than as a blocked action, since blocking it would remove the evidence that the failure was ever caught.

6.2 Limitations

- Single-user, single-browser scope by default: data is stored in localStorage, tied to one browser on one device. The Export/Import feature (Section 4.6) allows a case file to be manually moved between devices or backed up, but this is still a manual step, not real-time multi-user synchronisation. A production deployment would need shared, authenticated storage.
- Log tampering is now detectable, but not prevented: the hash-chaining mechanism in Section 4.4 means an edit to a past custody log entry is reliably caught by Verify Log Integrity, which is a meaningful improvement over the original design. However, the tool does not stop someone with direct access to browser storage from editing the log — it only guarantees that the edit will be visible on the next verification. A production system would additionally want the record hashes signed with a private key held only by an authenticated investigator, so that a broken chain could also be attributed to a specific person.
- No identity verification: the investigator sign-in screen (Section 4.6) improves usability by auto-filling names consistently, but it is explicitly not authentication. It does not verify that the person named actually performed the action, and the tool discloses this limitation to the user directly rather than implying a security guarantee that does not exist.
- Client-side only: there is no server-side audit copy beyond what a user manually exports, so if the local browser data is lost (cleared cache, different device) without a prior export, the record is lost with it.

6.3 Recommendations

For anyone extending this tool beyond a coursework exercise, three improvements would most directly address the limitations above:

- Add a lightweight backend (even a simple database-backed API) so custody records are stored centrally rather than per-browser, with the Export/Import feature (Section 4.6) serving as a natural migration path for existing case data into that backend.
- Extend the hash-chaining mechanism in Section 4.4 with per-investigator digital signatures (e.g., a keypair per user), so that a broken chain can be attributed to a specific person rather than only detected.

- Support export to a standard interchange format, such as a signed PDF/A or a structured format compatible with existing case management tools, so records can move between systems without losing their evidentiary value.

7. Conclusion

This project set out to close a practical gap between the formal chain-of-custody requirements described in standards such as NIST SP 800-86 and ISO/IEC 27037, and the informal, unverifiable tools — spreadsheets and paper forms — that small teams and student exercises typically rely on instead. The resulting tool is a lightweight, browser-only application that requires no server or database, yet implements the feature that actually matters most: cryptographic proof, via SHA-256 hashing, that an evidence file has or has not been altered between custody transfers.

Testing confirmed that the tool correctly computes and re-verifies hashes, correctly distinguishes a genuine match from a deliberate alteration, persists records across sessions, and produces a clean, printable custody report. Its limitations — single-browser storage, no identity verification, and no protection against tampering with the log itself — are exactly the kind of gaps a production deployment would need to close, and are laid out explicitly in Section 6 rather than glossed over.

More broadly, the project demonstrates that meaningful evidentiary integrity checking does not require expensive commercial forensic software: a browser's native cryptography API, combined with a disciplined data model based on recognised standards, is sufficient to build a tool that does more than look official — it actually catches tampering when it happens.

References

- [1] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to Integrating Forensic Techniques into Incident Response," NIST Special Publication 800-86, National Institute of Standards and Technology, Gaithersburg, MD, USA, 2006.
- [2] ISO/IEC 27037:2012, Information technology — Security techniques — Guidelines for identification, collection, acquisition and preservation of digital evidence, International Organization for Standardization, Geneva, Switzerland, 2012.
- [3] Association of Chief Police Officers (ACPO), "ACPO Good Practice Guide for Digital Evidence," Version 5, 2012.
- [4] D. Brezinski and T. Killalea, "Guidelines for Evidence Collection and Archiving," RFC 3227, Internet Engineering Task Force, 2002.
- [5] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.
- [6] W3C, "Web Cryptography API," W3C Recommendation, 2017. [Online]. Available: <https://www.w3.org/TR/WebCryptoAPI/>

Appendix A: Full Source Code

The complete source code for the Chain of Custody tool (a single HTML file containing all HTML, CSS, and JavaScript) is included in the accompanying GitHub repository at the URL given on the cover page, and is reproduced in full in the repository's Custody.html file. It is omitted here to keep the printed report within a reasonable length; see the repository README for instructions on running it locally.